NETRAX - Final Round Build

What was built for each requirement, how to set it up, how to test it

SEE. SENSE. SAFEGUARD. Sections 1-12 of the Elaborated System Requirements + acceptance tests 01-14

Where we are

Every software requirement in the brief is now built and installed on the laptop (C:\Users\Satyabrata\Desktop\sentineleye). A backup of every file that changed is in **_to_delete/before_final_update/**. An automatic acceptance test runs all 14 checklist tests with fake Telegram / Twilio / mail servers and passes: **ALL FINAL TESTS PASSED** (56 checks). The old self-tests still pass too.

Still needs you (live parts): (1) create the Telegram bot, Twilio trial and Gmail app password and put them in config/secrets.local.yaml; (2) plug in the USB webcam for the entrance; (3) run the live tests in section 5 and the latency measurement in section 6 so the numbers in the presentation are measured on our laptop. Hardware (GSM module, door lock) stays for later, as you asked.

What changed for PPE detection and speed (your main complaint)

Problem	What we changed	Where
Lag on every camera	ONE shared AI engine: each model is loaded once (not once per camera) and the newest frames of all cameras are processed together in ONE GPU call. Frames older than 0.6 s are skipped, so the system never works on a stale picture. Boxes are drawn only when a preview frame is due; JPEG encoding moved to its own thread; live view 10 frames/s with the capture time attached.	edge/engine.py, worker.py, publisher.py
Camera delay / blur	USB / laptop webcams: MJPG 1280x720 at 30 fps with a 1-frame driver buffer; network cameras keep only the newest frame. Optional fixed exposure to reduce motion blur.	edge/stream.py, cameras.yaml capture:
Which PPE is missing is unclear	Every worker box now lists each required item on its own line: green 'HELMET OK', red 'NO GLOVES', grey 'GOGGLES ?' (body part not visible, never counted as missing). The list is smoothed over the last ~8 frames so it does not flicker.	edge/annotate.py, worker._smooth
More speed if needed	Optional TensorRT engines (FP32, safe on GTX 1650): python -m tools.export_engines, then models.runtime: tensorrt. Falls back to .pt automatically.	tools/export_engines.py
Proof	Latency is measured, not guessed: median and 95th percentile per camera and end to end, on the dashboard and in docs/eval/latency_report.md.	backend/perf.py, tools/latency_report.py

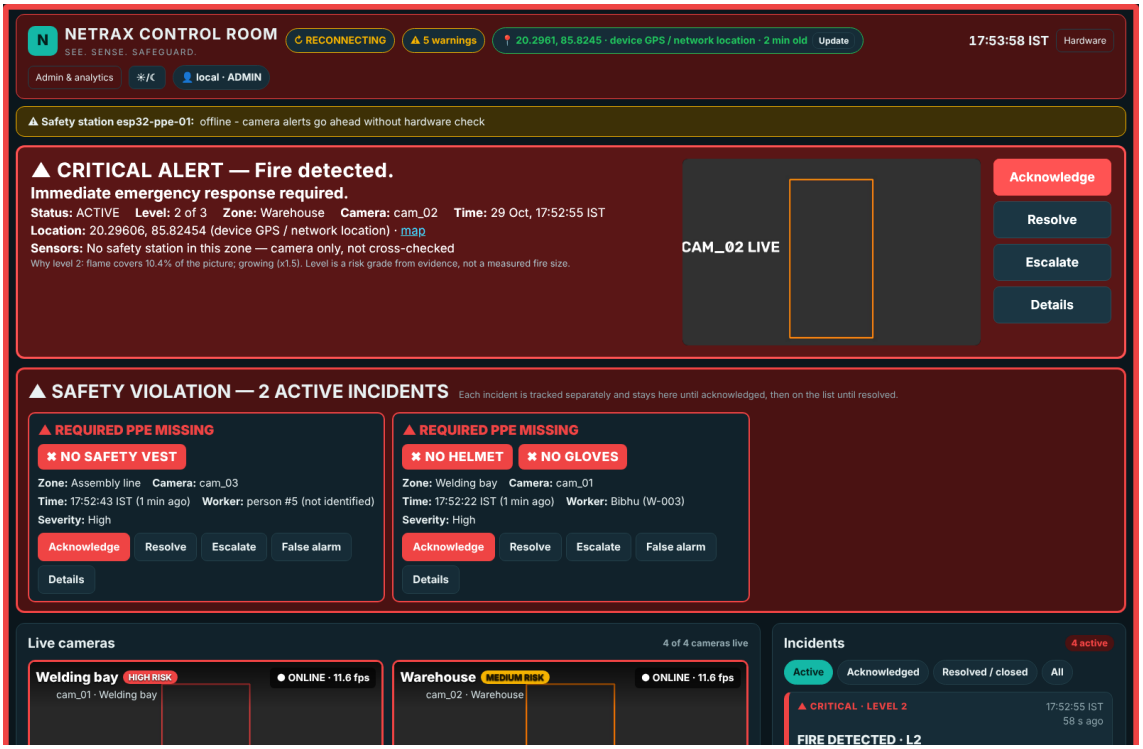


Worker overlay on the live picture (test image): one clear line per required item.

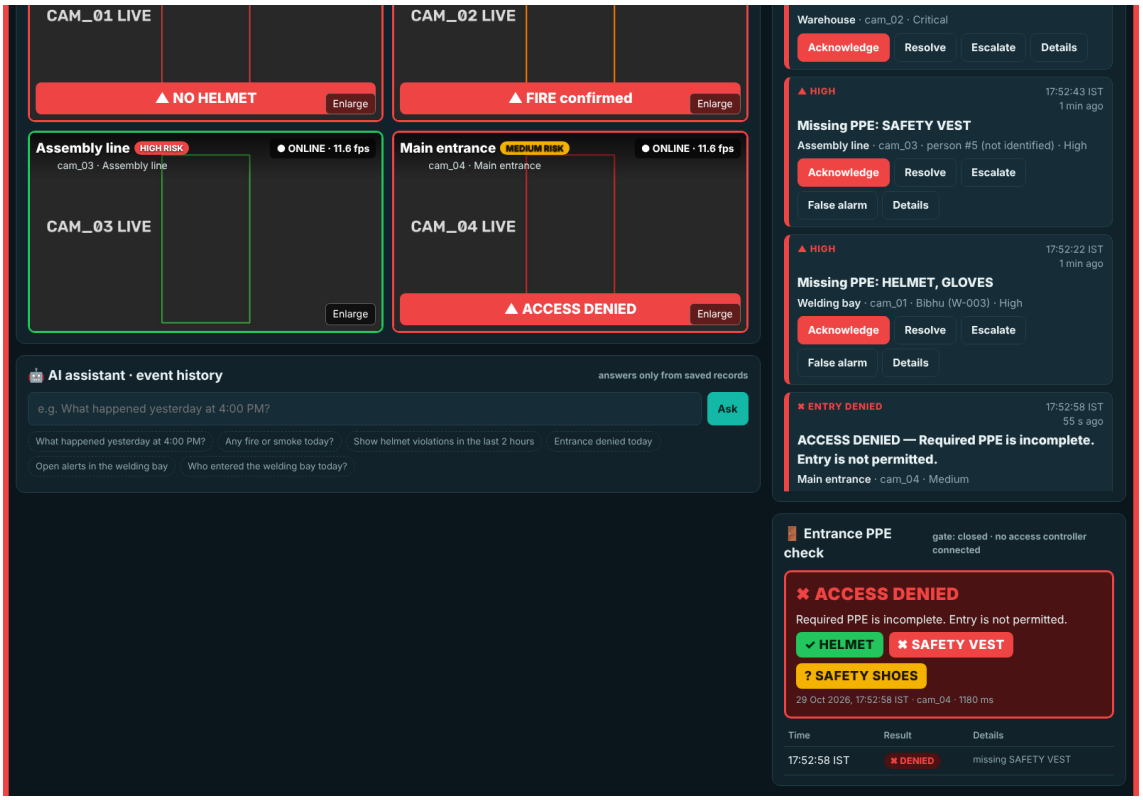
1. Requirement-by-requirement: what is built

#	Requirement	What NETRAX does now	Status
1	Dashboard UI/UX, big feeds	New Control Room at / : large 16:9 camera wall (2 x 2, 'Enlarge' to full width), 16-28 px text, states always text + icon + colour (NORMAL, WARNING, CRITICAL, EMERGENCY, ACKNOWLEDGED, RESOLVED), picture age + fps on every tile, health / location / clock in the header. Day / night switch.	Done
2	Full-screen violation	Active unacknowledged High/Critical PPE or access incident -> whole dashboard turns red + a large panel with one card PER incident (missing items, camera, zone, time, worker if identified). Stays until acknowledged; acknowledged != resolved (separate buttons, who + when recorded).	Done
3	AI assistant over history	Ask in plain English ('What happened yesterday at 4:00 PM?'). The question becomes a database search (date, time window, type, PPE item, zone, camera, worker, status); the answer is built only from the records found, each with snapshot and 'open record'. No match -> 'No matching record was found'. Times in IST; the searched window is always shown. Viewer role: names hidden. Every question logged.	Done (offline)
4	Entrance PPE check	4th camera (USB, role: entrance). One decision per person after >= 6 frames and >= 1 s: ACCESS APPROVED / ACCESS DENIED (names the missing item) / 'Verification failed - manual check required' (unclear, occluded, too far, 2 people). Big popup + voice. Supervisor records the manual check. Every decision stored with required items, evidence and time. Gate stays 'closed' unless approved AND a controller is connected.	Done (lock = later)
5	Telegram + phone + SMS + e-mail	Telegram bot (with snapshot), Twilio voice call + SMS, Gmail SMTP with snapshot. Background queue, bounded retries, one message per incident per channel, fire never delayed. Status shown only from the provider's answer (DELIVERED / SENT / FAILED / NOT CONFIGURED).	Built - needs your keys
6	GMS (hardware) + GPS (software)	Software side: incident location = hardware GPS (endpoint ready) > this device's GPS (browser) > configured site position; stale / missing shown as 'Location unavailable'; zone + camera labels always. Hardware GSM/GPS module: later.	Software done
7	Latency	Shared batched engine, stale-frame skip, lazy drawing, threaded encoding, MJPG capture; p50/p95 for every stage (edge, server, dashboard, providers).	Done - measure live
8	Big fire / smoke panel	FIRE DETECTED / SMOKE DETECTED panel with time, camera, zone, level, status, live feed, location, sensor values and whether the station confirms; 'verifying' shown as not yet confirmed.	Done
9	Fire levels 1-3	Level from flame area, growth, duration, station flame / heat / smoke and number of sources. Level never goes down during an incident. L1 yellow + voice, L2 red + siren + Telegram + call, L3 full-screen EVACUATE + every channel. Exact messages from the brief.	Done (thresholds = prototype)
10	Database, reliability	All checks, decisions, detections, alerts and delivery attempts stored with one event ID; search by date/type/worker/camera/zone/severity/status; login with roles; audit log; health panel (edge, each camera, DB, each channel, queue, station, location, disk); offline outbox unchanged.	Done
11	Acceptance tests 01-14	python -m tools.final_selftest (automatic) + the live script in section 5.	Auto: PASS
12	Dependencies	GMS = GSM module (hardware, later); phone = Twilio voice call; PPE per zone in zones.yaml.	Decided

2. The new Control Room (screenshot with test data)



Top: header (overall state, health, location, clock, user/role), health warning strip, Level 2 fire panel with the affected camera, and the violation panel with one card per active incident.



Middle: camera wall with status, fps and badges; incident list (Active / Acknowledged / Resolved); entrance result; AI assistant. Below (not shown): zones, alert delivery, measured latency, system health. The old dashboard (workers, reports, analytics) is at /admin.

3. One-time setup (about 20 minutes)

3.1 Update the Python packages (venv)

```
cd C:\Users\Satyabrata\Desktop\sentineleye
venv\Scripts\activate
pip install tzdata
python -m tools.final_selftest        (must end with ALL FINAL TESTS PASSED)
python -m edge.selftest               (ALL TESTS PASSED)
```

3.2 Alert channels: create config\secrets.local.yaml

Copy **config\secrets.example.yaml** to **config\secrets.local.yaml** and fill it in. This file is in .gitignore - never push it or paste its contents anywhere.

Channel	Steps	Check
Telegram	Telegram app -> @BotFather -> /newbot -> copy the token into telegram.bot_token. Create group 'NETRAX Alerts', add the bot, send 'hi' in the group. Run **python -m tools.notify_setup telegram** and copy the negative chat_id into telegram.chat_id.	python -m tools.notify_setup test telegram
Phone call + SMS (Twilio)	twilio.com -> free trial. Verify each phone that should receive alerts (Console -> Phone Numbers -> Verified Caller IDs). Get a Twilio number. Copy Account SID, Auth Token, the Twilio number and the verified numbers (+91...) into the twilio section.	... test call sms (your phone rings and reads the alert)
E-mail (Gmail)	Sender Google account -> Security -> 2-Step Verification ON -> 'App passwords' -> create 'NETRAX' (16 letters). username = that Gmail, app_password = 16 letters, to = list of receivers.	... test email

Twilio trial note: trial calls start with a short Twilio message and only reach verified numbers - say so if asked. Indian SMS through a trial can be filtered by the operator; the dashboard then shows SENT or FAILED with the reason, never DELIVERED without proof.

3.3 Login

Start the backend once (run_all.bat). It creates three users and writes the passwords to **data\initial_passwords.txt**. Set your own:

```
python -m tools.users set admin <new-password> admin
python -m tools.users set supervisor <new-password> supervisor
python -m tools.users set jury <password> viewer          (read-only demo account, names hidden)
```

3.4 Entrance camera + location

Plug the USB webcam in (cameras.yaml cam_04, source: 1). Mount it 2-3 m in front of the door at chest height so the whole body - helmet to shoes - is in the picture; one person at a time. Required items are in zones.yaml -> main_entrance (helmet, vest, boots). For location, either click 'Use this device' in the header on the laptop (browser GPS works on http://127.0.0.1:8000) or enter the measured site position in config\site.yaml (site.lat / site.lon, from a Google Maps pin). Never type invented coordinates.

3.5 Phones and Wi-Fi

python -m tools.find_phones --write as before. For alert links on phones, set dashboard_url in config\notify.yaml to the laptop's Wi-Fi address (e.g. http://172.20.163.235:8000).

4. Run it

```
run_all.bat            (backend + edge, opens http://127.0.0.1:8000 -> sign in)
Control room:   http://127.0.0.1:8000/
Admin / workers / reports:   http://127.0.0.1:8000/admin
Hardware:       http://127.0.0.1:8000/hardware
```

The edge window should print: *shared inference engine: models loaded once, batch up to 4 cameras* and *[cam_04] ENTRANCE check for zone main_entrance: helmet, vest, boots*.

Better PPE results on camera (do these before the demo)

• Light: bright, even light on the workers' front. Backlight (window behind) is the biggest cause of missed helmets.
• Distance: 2-5 m. A worker must be at least ~1/3 of the picture height for gloves/goggles; shoes need the feet in view.
• Phones: IP Webcam app at 1280x720, quality 70, phone on a stand (not in the hand), rotate set in cameras.yaml.
• Re-register every member with 3-5 photos (front, left, right, with helmet) on /admin - wrong names come from too few photos.
• Real PPE: hi-vis vest (lime/orange), hard hat (not a cap). Grey 'ITEM ?' means 'body part not visible' - turn to the camera.

5. Live acceptance tests (record result + screenshot for each)

ID	Do this	Expected (where to look)
01	Person with helmet + vest + safety shoes stands in front of cam_04 for 2 s.	Green popup ACCESS APPROVED; Entrance panel + /admin history show it.
02	Same person without the vest.	Red popup ACCESS DENIED - 'SAFETY VEST' marked missing.
03	Stand too close / hide feet / two people together.	Amber 'Verification failed - manual check required'; supervisor clicks 'Checked - approve/deny'.
04	Worker removes helmet in the Welding bay (cam_01).	Whole dashboard red, violation card names HELMET; Acknowledge -> blue; Resolve -> closed with your name.
05	Lighter / candle in front of a phone camera (safely).	Big FIRE DETECTED panel: time, camera, zone, level, status, sensor line.
06	Keep it burning / bring it closer (or run the self-test).	Level 1 yellow -> Level 2 red + siren -> Level 3 full screen; never goes down.
07	Any High alert with Telegram set up.	Message in the group with snapshot; Alert delivery row DELIVERED.
08	Critical alert (fire Level 2).	Verified phone rings and reads the alert; row DELIVERED after the call ends.
09	Same alert.	SMS + e-mail with type, severity, zone, time, action, event ID; retries visible if a try fails.
10	Ask: 'What happened today at ?'	The helmet incident with snapshot and 'open record'.
11	Two workers without PPE on two cameras at once.	Two separate cards; acknowledging one keeps the other.
12	tools.latency_report with 1 camera, then all 4.	Table in docs\eval\latency_report.md (median + 95th %).
13	Turn Wi-Fi/internet off, raise an alert.	Dashboard still shows the alert at once; Telegram row RETRYING then FAILED; health strip red.
14	Location: none, then 'Use this device', then wait.	'Location unavailable' -> coordinates with source -> 'stale' after 15 min.

6. Measure the latency (numbers for the jury)

Keep the dashboard open on the laptop (it reports picture age and event arrival), walk in front of the cameras, remove a helmet once or twice, then run:

```
python -m tools.latency_report --label "1 camera (cam_01 only)" --seconds 120
python -m tools.latency_report --label "4 cameras + dashboard" --seconds 180
```

For the 1-camera run set enabled: false on cam_02..cam_04 in cameras.yaml (put them back after). Optional: **pip install tensorrt**, **python -m tools.export_engines**, models.runtime: tensorrt, run the report again and keep whichever is faster. Quote only these measured values; 'time to alert' includes the deliberate 1-2 s confirmation that stops false alarms - say that.

What each number means

Number	Meaning
capture -> processed (per camera)	camera frame taken -> all models + logic done on the edge
frame captured -> event stored / on dashboard	how long an alert takes to reach the server / the screen
live picture age	how old the picture on the dashboard is when you see it
hazard first seen -> alert	includes the confirmation window (one frame is never enough)
Telegram / call / SMS / e-mail	until the provider confirmed (call = answered and finished)

7. Settings you may tune

File -> key	Default	Effect
settings.yaml processing.target_fps	12	upper limit per camera; the GPU decides the real rate
settings.yaml processing.max_frame_age_s	0.6	older frames are skipped (no lag build-up)
settings.yaml entrance.min_look_s / max_look_s	1.0 / 4.0	fastest decision / when to ask for a manual check
settings.yaml fire_levels.*	prototype	area, growth, duration, temperature for levels 2 and 3
settings.yaml incidents.emergency_severities	Critical, High	which PPE incidents turn the dashboard red
notify.yaml rules	see file	which incident goes to which channel
zones.yaml main_entrance.required_ppe	helmet, vest, boots	items checked at the entrance

8. Honest limits (say them before the jury asks)

• Fire level thresholds are prototype values from our tests; the level is a risk grade, not a measured fire size.
• Door lock / turnstile and the GSM module are not connected yet: the gate decision is advisory and shown as such.
• Twilio trial: only verified numbers; Indian SMS may be filtered - the dashboard shows what the provider reported.
• The AI assistant is a grounded query engine, not a free chat model: it cannot answer questions outside the stored records (by design).
• Speed numbers depend on the laptop GPU (GTX 1650); quote the measured report, not estimates.

9. If something goes wrong

Symptom	Fix
Dashboard asks to sign in again	Session lasts 12 h; sign in. Lost password: python -m tools.users set admin admin
Health: Alerts NOT CONFIGURED	secrets.local.yaml missing a value; python -m tools.notify_setup status
Telegram 'Unauthorized'	Wrong bot token. 'chat not found': run notify_setup telegram again after a group message
Entrance always MANUAL	Feet or head out of the picture, or the person too far: move the camera back / lower it
cam_04 OFFLINE	USB webcam index differs: try source: 2 in cameras.yaml

Symptom	Fix
Edge prints 'inference engine did not answer'	GPU busy / crashed: restart run_all.bat; set processing.shared_engine: false to fall back to the old mode
Roll back everything	copy the files from _to_delete\before_final_update\ back (same folders)